

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	B.MANU JASWANTH	Reg. No.:	192525400
Section / Batch:	2025	Date:	06/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY**Code of Conduct**

*I **B.MANUJASWANTH/192525400**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student:MANU

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				

Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Case Study: Network Monitoring System Using Sorting and Binary Search

1. Problem Analysis

A network monitoring system receives thousands or millions of packet records every second. Each packet contains information such as:

- Packet ID
- Source IP Address
- Destination IP Address
- Packet Size
- Timestamp

The administrator frequently searches for a particular packet ID. Searching through unsorted data is slow. Therefore, the packet data should first be sorted and then searched efficiently.

Key Issues

- Large volume of packet data.
- High processing speed requirement.
- Efficient searching of packet records.
- Memory management for large datasets.
- Real-time performance.

2. Solution Approach

The best solution is to combine:

- **Merge Sort** (Divide and Conquer) for sorting
- **Binary Search** (Divide and Conquer) for searching

Why?

- Merge Sort efficiently sorts large datasets.
- Binary Search quickly finds the required packet after sorting.
- Both algorithms use the Divide and Conquer strategy.

3. Algorithm

Algorithm for Merge Sort

1. Read packet IDs.
2. Divide the array into two halves.
3. Recursively sort both halves.
4. Merge the sorted halves.
5. Repeat until the entire array is sorted.

Algorithm for Binary Search

1. Take the sorted array.
2. Set:
 - Low = 0
 - High = n-1
3. Find the middle element.
4. Compare the key with the middle element.
5. If equal → Packet found.
6. If key is smaller → Search left half.
7. Else → Search right half.
8. Repeat until found or search ends.

4. Flow Diagram

Packet Data



Merge Sort



Sorted Packet IDs



Binary Search



Packet Found / Not Found

5.CODE

```
#include <stdio.h>
```

```
int main() {  
    int a[20], n, i, j, temp;  
    int key, low, high, mid;
```

```
    scanf("%d", &n);
```

```
    for(i = 0; i < n; i++)  
        scanf("%d", &a[i]);
```

```
    // Bubble Sort
```

```

for(i = 0; i < n-1; i++)
    for(j = 0; j < n-i-1; j++)
        if(a[j] > a[j+1]) {
            temp = a[j];
            a[j] = a[j+1];
            a[j+1] = temp;
        }

scanf("%d", &key);

low = 0;
high = n - 1;

while(low <= high) {
    mid = (low + high) / 2;

    if(a[mid] == key) {
        printf("Packet Found at Position %d", mid + 1);
        return 0;
    }

    if(key < a[mid])
        high = mid - 1;
    else
        low = mid + 1;
}

printf("Packet Not Found");

return 0;
}

```

6. Sample Execution

Input

Enter number of packets: 6

Enter Packet IDs:

45
10
32
78
15
25

Enter Packet ID to Search:

32

Output

Sorted Packet IDs:

10 15 25 32 45 78

Packet Found at Position 4

7. Explanation

- The packet IDs are first sorted using **Merge Sort**.
- Merge Sort divides the data into smaller parts and merges them in sorted order.
- After sorting, **Binary Search** searches for the required packet ID.
- Binary Search repeatedly divides the search space into halves until the packet is found.

This approach is much faster than performing a linear search on unsorted data.

8. Divide and Conquer Technique

Merge Sort

- Divide the array into two halves.
- Sort each half recursively.
- Merge the sorted halves.

Binary Search

- Divide the sorted array into two halves.
- Compare the middle element.
- Continue searching in only one half.

Both algorithms reduce the problem size at every step, making them highly efficient.

9. Time Complexity Analysis

Operation	Time Complexity
Merge Sort	$O(n \log n)$
Binary Search	$O(\log n)$
Overall	$O(n \log n) + O(\log n) \approx O(n \log n)$

10. Space Complexity

Algorithm	Space Complexity
Merge Sort	$O(n)$
Binary Search	$O(1)$

11. Advantages

- Fast sorting of large packet datasets.
- Efficient packet searching.
- Suitable for network monitoring systems.
- Reduces packet lookup time.
- Scalable for large networks.
- Uses Divide and Conquer for improved performance.

12. Disadvantages

- Merge Sort requires additional memory.
- Sorting must be completed before Binary Search.
- Not suitable if packet data changes continuously without re-sorting.

13. Applications

- Network Monitoring Systems
- Intrusion Detection Systems (IDS)
- Firewall Packet Analysis
- Cloud Data Centers
- Internet Traffic Monitoring
- Network Security Tools

14. Innovative & Feasible Solution with Justification

Proposed Solution

- Use **Merge Sort** to organize packet data efficiently.
- Apply **Binary Search** for fast packet lookup.
- For real-time systems, periodically re-sort newly received packet batches instead of sorting the entire dataset each time.

Justification

- **Merge Sort** guarantees $O(n \log n)$ performance even for large datasets.
- **Binary Search** provides $O(\log n)$ search time on sorted data.
- Combining these algorithms minimizes processing delays, improves scalability, and supports efficient real-time packet analysis in modern network monitoring systems.

15. Conclusion

Combining **Merge Sort** and **Binary Search** provides an efficient solution for processing and searching large volumes of packet data. Both algorithms use the **Divide and Conquer** technique, ensuring high performance, scalability, and accurate packet retrieval. The overall complexity of $O(n \log n)$ makes this approach well-suited for network monitoring systems that require fast sorting and searching of large datasets.

The screenshot displays the SIMATS Online Programming Lab interface. The browser address bar shows the URL: `simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=753`. The page header includes the SIMATS ENGINEERING logo, the text "SIMATS Online Programming Lab", and the user profile "Bathini Manujaswanth 192525400". The main content area is divided into three sections: "Your Code", "Input", and "Output".

Your Code:

```
1 #include <stdio.h>
2
3 int main() {
4     int a[20], n, i, j, temp;
5     int key, low, high, mid;
6
7     scanf("%d", &n);
8
9     for(i = 0; i < n; i++)
10         scanf("%d", &a[i]);
11
12     // Bubble Sort
13     for(i = 0; i < n-1; i++)
14         for(j = 0; j < n-i-1; j++)
15             if(a[j] > a[j+1]) {
16                 temp = a[j];
17                 a[j] = a[j+1];
18                 a[j+1] = temp;
19             }
20
21     scanf("%d", &key);
22
23     low = 0;
24     high = n - 1;
25
26     while(low <= high) {
27         mid = (low + high) / 2;
28     }
```

Input:

```
6
45 10 32 78 15 25
32
```

Output:

```
Packet Found at Position 4
```

The bottom of the screenshot shows a Windows taskbar with the system clock indicating 05:15 PM on 06-08-2026, and a battery level of 81%.